P.Vishwa Nandan(192473015)

Q6 – Priority, Order and Fairness in a Queue

Marks: 8

1. Introduction

A hospital emergency department receives patients with different medical severity levels. The hospital must treat patients according to their medical priority.

The following rules are used:

1. Severity 1 represents critical patients.
2. Severity 5 represents minor patients.
3. Patients with lower severity numbers must be treated first.
4. If two patients have the same severity, the patient who arrived first must be treated first.

A PriorityQueue is suitable for this system because it processes elements according to priority.

2. Why Is a Normal Queue Not Sufficient?

A normal Queue follows the FIFO (First In, First Out) principle. The first patient who enters the queue will be treated first.

However, in a hospital, a critical patient must be treated before a minor patient, even if the minor patient arrived earlier.

Example:

Patient	Severity	Arrival Order
Ravi	3	1
Meena	1	2
John	5	3

A normal FIFO queue produces this order:

Ravi → Meena → John

But the correct emergency treatment order is:

Meena → Ravi → John

Therefore, a normal Queue is not suitable because it cannot automatically arrange patients according to medical severity.

A PriorityQueue is more suitable for this requirement.

3. PriorityQueue with Comparator

PriorityQueue is a Java collection that stores elements according to their priority.

In our hospital system:

- Lower severity number means higher priority.
- Severity 1 is treated first.
- Severity 5 is treated last.

A Comparator is used to define the priority order.

```
Comparator<Patient> priorityOrder =
   Comparator.comparingInt((Patient p) -> p.severity)
         .thenComparingLong(p -> p.arrival);
```

Here:

1. Severity arranges patients from the lowest number to the highest number.
2. Arrival order arranges patients with equal severity.

Therefore, critical patients are treated first, and patients with equal severity are treated according to their arrival order.

4. How Arrival Order Is Preserved

PriorityQueue does not automatically maintain FIFO order for elements having equal priority.

Therefore, we introduce an arrival counter.

Example:

Patient	Severity	Arrival Counter
Ravi	3	0
Meena	1	1
John	3	2
Asha	5	3
Kumar	1	4
Lata	2	5

The arrival counter increases whenever a new patient is admitted.

When two patients have the same severity, the Comparator checks their arrival counters.

The patient with the smaller arrival counter is treated first.

This preserves arrival order among patients with equal severity.

5. Queue Design and Working

The hospital triage system contains the following components.

Patient Class:

Stores the patient name, severity level and arrival counter.

Triage Class:

Manages the PriorityQueue and contains the following methods:

- admit() – Adds a patient to the queue.
- next() – Removes and returns the highest-priority patient.
- displayWaitingList() – Displays patients in treatment order.

PriorityQueue:

Stores patients according to severity and arrival order.

Working Process

1. A patient arrives at the hospital.
2. The system assigns a severity level.
3. An arrival counter is assigned.
4. The patient is inserted into PriorityQueue.
5. The doctor calls next() to get the highest-priority patient.
6. The patient is removed from the queue.

6. Complete the Code – PriorityQueue Implementation

The assignment provides one incomplete section.

TODO 1: Create a PriorityQueue that serves the lowest severity number first and, for equal severity, the earliest arrival first.

Java Program

File name: Triage.java

```
import java.util.Comparator;

import java.util.PriorityQueue;
```

```
class Patient {

    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    @Override
    public String toString() {
        return name + "(S" + severity + ")";
    }
}

public class Triage {

    private long counter = 0;

    // PriorityQueue with severity and arrival-order comparison
    private PriorityQueue<Patient> queue =
            new PriorityQueue<>(
                Comparator.comparingInt(
                    (Patient p) -> p.severity
                ).thenComparingLong(
```

```
                p -> p.arrival
            )
        );

// Admit a patient
void admit(String name, int severity) {

    if (severity < 1 || severity > 5) {
        throw new IllegalArgumentException(
            "Severity must be between 1 and 5"
        );
    }

    queue.add(
        new Patient(name, severity, counter++)
    );
}

// Remove the highest-priority patient
Patient next() {
    return queue.poll();
}

// Display patients in treatment order
void displayWaitingList() {

    PriorityQueue<Patient> copy =
            new PriorityQueue<>(queue);
```

```
        while (!copy.isEmpty()) {
            System.out.println(copy.poll());
        }
    }

    public static void main(String[] args) {

        Triage er = new Triage();

        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);

        StringBuilder order = new StringBuilder();

        Patient p;

        while ((p = er.next()) != null) {
            order.append(p).append(' ');
        }

        System.out.println(order.toString().trim());
    }
}
```

7. Expected Output

Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)

Explanation of Output

First – Meena(S1)

Meena has severity 1, so she is treated first.

Second – Kumar(S1)

Kumar also has severity 1 but arrived after Meena. Therefore, Meena is treated first.

Third – Lata(S2)

Lata has severity 2.

Fourth – Ravi(S3)

Ravi has severity 3 and arrived before John.

Fifth – John(S3)

John has the same severity as Ravi but arrived later.

Sixth – Asha(S5)

Asha has severity 5, so she is treated last.

How Does the Arrival Counter Work?

The arrival counter gives every patient a unique sequence number.

When two patients have the same severity, the Comparator checks their arrival values.

The smaller arrival value receives higher priority.

Therefore, patients with equal severity are treated in their original arrival order.

8. Eight-Patient Correctness Test

The assignment requires a small test with approximately eight patients whose treatment order can be verified manually.

Consider the following patients:

Patient	Severity	Arrival Order
Ravi	3	1
Meena	1	2
John	3	3
Asha	5	4
Kumar	1	5

Lata	2	6
Nila	2	7
Omar	4	8

The following statements are added to the program:

er.admit("Nila", 2);

er.admit("Omar", 4);

Expected Treatment Order

Meena(S1)

Kumar(S1)

Lata(S2)

Nila(S2)

Ravi(S3)

John(S3)

Omar(S4)

Asha(S5)

Explanation

1. Meena and Kumar have severity 1. Meena arrived first.
2. Lata and Nila have severity 2. Lata arrived first.
3. Ravi and John have severity 3. Ravi arrived first.
4. Omar has severity 4.
5. Asha has severity 5.

This output satisfies both priority conditions.

Therefore, the PriorityQueue design works correctly for the eight-patient test.

9. Why Printing PriorityQueue Directly Does Not Show Treatment Order

Consider:

System.out.println(queue);

This prints the internal arrangement of the PriorityQueue.

It does not guarantee that all patients appear in treatment order.

A PriorityQueue guarantees that its highest-priority element is available through peek() and poll(), but its iterator does not provide sorted traversal.

To display the waiting list correctly, we create a copy and repeatedly remove patients from that copy.

Example:

```
PriorityQueue<Patient> copy =
        new PriorityQueue<>(queue);


while (!copy.isEmpty()) {
   System.out.println(copy.poll());
}
```

This displays patients in their correct treatment order without modifying the original queue.

10. Growth Investigation and Complexity Analysis

The assignment asks us to predict the cost of adding a patient and removing the next patient.

A. Adding a Patient

When a patient is added using:

queue.add(patient);

PriorityQueue places the patient in the correct heap position.

Time complexity: **O(log n)**.

B. Removing the Next Patient

When the next patient is removed using:

queue.poll();

PriorityQueue removes the highest-priority element and rearranges the heap.

Time complexity: **O(log n)**.

C. Other Operations

Operation	**Time Complexity**
Add patient	O(log n)
Remove next patient	O(log n)
View next patient using peek()	O(1)

Display all patients in priority order	O(n log n)
Search for a particular patient	O(n)

Space complexity: **O(n)** because the queue stores n patient objects.

D. Experimental Workloads

We use three different queue sizes:

- Experiment 1: 1,000 patients.
- Experiment 2: 10,000 patients.
- Experiment 3: 50,000 patients.

We measure:

1. Time required to admit all patients.
2. Time required to serve all patients.

Java Benchmark Program

File name: QueueBenchmark.java

```
import java.util.Comparator;
import java.util.PriorityQueue;


class BenchPatient {


   int severity;
   long arrival;


   BenchPatient(int severity, long arrival) {
      this.severity = severity;
      this.arrival = arrival;
   }
}


public class QueueBenchmark {
```

```
static volatile long result;

static void test(int n) {

    int repetitions = 5;

    long totalAddTime = 0;
    long totalRemoveTime = 0;

    for (int r = 0; r < repetitions; r++) {

        PriorityQueue<BenchPatient> queue =
                new PriorityQueue<>(
                    Comparator.comparingInt(
                        (BenchPatient p) -> p.severity
                    ).thenComparingLong(
                        p -> p.arrival
                    )
                );

        // Measure patient admission time
        long start = System.nanoTime();

        for (int i = 0; i < n; i++) {

            int severity = (i % 5) + 1;

            queue.add(
                new BenchPatient(severity, i)
```

```
            );
        }

        long addEnd = System.nanoTime();

        // Measure patient serving time
        while (!queue.isEmpty()) {

            BenchPatient p = queue.poll();

            result += p.arrival;
        }

        long removeEnd = System.nanoTime();

        totalAddTime += addEnd - start;
        totalRemoveTime += removeEnd - addEnd;
    }

    System.out.println("\nPatients: " + n);

    System.out.printf(
        "Average admission time: %.3f ms%n",
        totalAddTime / (double) repetitions / 1_000_000
    );

    System.out.printf(
        "Average serving time: %.3f ms%n",
        totalRemoveTime / (double) repetitions / 1_000_000
```

```
        );
    }

    public static void main(String[] args) {

        test(1000);
        test(10000);
        test(50000);

        System.out.println("\nExperiments completed.");
    }
}
```

Experimental Results

Record the actual timings from your Java execution.

Number of Patients	Admission Time	Serving Time
1,000	Actual result	Actual result
10,000	Actual result	Actual result
50,000	Actual result	Actual result

The timings may vary depending on the computer and Java environment.

Predicted Behaviour:

As the number of patients increases, admission and removal times increase according to O(log n) per operation.

PriorityQueue is suitable for handling large numbers of patients because it does not need to sort the entire collection after every admission.

11. Starvation and Fairness

Starvation occurs when a low-priority patient waits for a very long time because higher-priority patients continuously arrive.

For example, a patient with severity 5 may never receive treatment if new severity 1, 2, 3 and 4 patients continuously enter the queue.

Solution – Priority Aging

We can introduce a technique called priority aging.

Priority aging gradually increases the priority of patients who have waited too long.

For example:

- A patient starts with severity 5.
- After waiting 30 minutes, the effective severity becomes 4.
- After another 30 minutes, the effective severity becomes 3.
- After another 30 minutes, the effective severity becomes 2.
- The effective severity cannot become less than 1.

Example formula:

```
int effectiveSeverity =
    Math.max(1, severity - (waitMinutes / 30));
```

Here, a lower effective severity represents higher priority.

If the priority of a patient changes, the PriorityQueue must be reorganised. We can periodically rebuild the queue using updated effective severities instead of changing comparator values while patients remain in the heap.

This reduces starvation while maintaining medical priority.

In a real hospital, medical staff should define and approve the aging rules so that they do not override clinically necessary treatment priorities.

12. Deployment Decision – City Hospital with 5,000 Patients per Day

The PriorityQueue design is suitable for the core triage operation of a hospital receiving 5,000 patients daily.

Reasons:

1. PriorityQueue supports efficient patient admission.
2. It removes the highest-priority patient in O(log n) time.
3. The arrival counter preserves order among patients with equal severity.
4. It can manage large numbers of waiting patients.
5. Its space complexity is O(n).

For 5,000 waiting patients, the heap height is approximately 13 levels because $\log_2(5000)$ is approximately 12.3.

Therefore, basic queue operations require relatively few heap adjustments.

However, a real city hospital also needs:

- A database to preserve patient information.
- Multiple doctors and emergency departments.
- Thread-safe queue operations.
- Patient history and treatment records.
- Proper clinical priority rules.
- Monitoring and audit facilities.

Java PriorityQueue is not thread-safe, so a concurrent implementation or suitable locking is needed when several hospital staff members access the queue simultaneously.

Final Decision: PriorityQueue is suitable for the core emergency triage algorithm, provided that database storage, concurrency management and clinical fairness rules are also implemented.

13. Alternative Concept and Complexity Analysis

The assignment asks us to compare PriorityQueue with two alternative approaches.

A. LinkedList Maintained in Sorted Order

Instead of using PriorityQueue, we can use a LinkedList and insert every patient in the correct position.

When a new patient arrives:

1. Traverse the list.
2. Find the correct priority position.
3. Insert the patient.
4. Remove the first patient when treatment begins.

Example:

LinkedList<Patient> patients = new LinkedList<>();

Complexity:

- Admission: O(n), because the correct insertion position must be found.
- Serving the next patient: O(1), because the first patient can be removed directly.
- Space: O(n).

B. Array of Five FIFO Queues

Since severity has only five fixed levels, we can create five separate FIFO queues.

Example:

Queue<Patient>[] queues = new Queue[5];

Each queue represents one severity level.

- Queue 1: Severity 1.
- Queue 2: Severity 2.
- Queue 3: Severity 3.
- Queue 4: Severity 4.
- Queue 5: Severity 5.

When a patient arrives, they are inserted into the corresponding queue.

When a doctor is ready, the system checks severity queues from 1 to 5 and serves the first available patient.

Complexity:

- Admission: O(1).
- Serving the next patient: O(1), because only five queues need to be checked.
- Space: O(n), plus fixed queue overhead.

C. Comparison

Operation	**PriorityQueue**	**Sorted LinkedList**	**Five FIFO Queues**
Admit patient	O(log n)	O(n)	O(1)
Serve next patient	O(log n)	O(1)	O(1)
Preserve equal-priority arrival order	Using arrival counter	Naturally	Naturally
Display in treatment order	O(n log n)	O(n)	O(n)
Space	O(n)	O(n)	O(n)

Recommendation

PriorityQueue is suitable when there are many patients with different priority levels because it maintains the highest-priority patient efficiently.

Five FIFO queues are also an excellent solution when the number of severity levels is fixed at five. They provide O(1) admission and treatment operations while naturally preserving arrival order within each severity level.

The selection depends on whether the priority levels are fixed or expected to change in the future.

14. Conclusion

PriorityQueue is suitable for hospital emergency triage because it automatically serves patients according to severity. A Comparator arranges patients with lower severity numbers first, while an arrival counter preserves FIFO order among patients with equal severity. Priority aging can help reduce starvation. The design provides efficient admission and removal operations and can be extended for larger hospital environments with suitable concurrency and database support.